Language Identification from Audio with Speaker Bias Mitigation.

Prem Kumar Vairamuthu (7075922) **Harjit Ravikanth Lakshmi (7075899)** **Hemashruthi Durairaj (7084037)**
Department of Computer Science, Saarland University, Saarbrücken, Germany
prva00001@stud.uni-saarland.de, hary00001@stud.uni-saarland.de, hedu00002@stud.uni-saarland.de

Abstract

In this project, we develop a system that classifies speech into 22 Indian languages using the badrex/nnti-dataset-full dataset. A major challenge is speaker bias since each language only has five speakers. We first reproduce the baseline model and apply different pre-trained speech models and hyperparameter tuning to improve performance. To reduce speaker bias, we use data-centric augmentation and model-based adversarial training. We then evaluate multiple speaker bias mitigation variants and find that DANN gives the best result: 0.5576 accuracy and 0.5426 macro-F1. We also analyze bias source, confusion patterns, and last-layer t-SNE behavior. Results show clear bias reduction and weak-class recovery, but not complete removal of speaker information.

1 Introduction

This project focuses on language identification with significant speaker variation. The main challenge is learning language features from a small number of training speakers while generalizing to unseen validation speakers.

Implementation uses PyTorch (Paszke et al., 2019), Hugging Face Transformers/Datasets (Wolf et al., 2020), the NNTI dataset card (Badr et al., 2026), MMS resources (Pratap et al., 2023; Meta AI (facebook), 2026), PyTorch Audio augmentation practice (PyTorch Audio Team, 2026), scikit-learn (Pedregosa et al., 2011), Matplotlib (Hunter, 2007), wav2vec2 (Baevski et al., 2020), HuBERT (Hsu et al., 2021), Domain-Adversarial Neural Network (DANN) (Ganin and Lempitsky, 2014), and t-SNE (van der Maaten and Hinton, 2008).

2 Dataset

We use `badrex/nnti-dataset-full` (Badr et al., 2026). Core split statistics:

- Languages: 22
- Train samples: 8689
- Validation samples: 3300
- Train unique speakers: 110
- Validation unique speakers: 681
- Train/validation speaker overlap: 0

The validation data comes from speakers that are different from those in the training data.

3 Experiments

3.1 Experimental Setup

To keep comparisons fair, we use the same dataset split and fixed random seed and changed only intended components per run. Shared protocol:

- Fixed split: same train/validation files across runs.
- Fixed seed: 42.
- Best-checkpoint loading enabled (`load_best_model_at_end=True`).
- Same diagnostics family: per-language accuracy, confusion matrix, speaker diagnostics.

3.2 Evaluation Metrics

We report:

- Global metrics: validation accuracy, macro-F1, eval loss.
- Class-level metric: per-language validation accuracy.
- Error structure: off-diagonal confusion pairs from confusion matrix.
- Speaker diagnostics: overlap count and unseen-speaker accuracy.
- Representation diagnostics: last-layer t-SNE (where artifacts are available).

Macro-F1 is prioritized over accuracy for evaluating speaker-bias mitigation.

4 Baseline Reproduction and Improvements

4.1 Methods

The baseline-improvement pipeline follows two stages:

- **Baseline reproduction**: reproduce the baseline by fine-tuning the `facebook/mms-300m` model using the provided starter script, aiming to reach the 31.96% top-1 accuracy
- **Improvement stage**: we experiment with other pretrained speech models and better hyperparameter tuning (e.g., learning rate, batch size, warmup steps, and training schedules). This results in a tuned mHuBERT reference model.

To exceed the baseline performance, we increased the number of training epochs and enabled dropout. we increased the number of training epochs, enabled dropout, set the learning rate to 3×10^{-5}, and used (`group_by_length=True`)

The main hyperparameter settings reported for this baseline-improvement stage are: effective batch control (per-device batch size around 16 with gradient accumulation), checkpointing policy (`load_best_model_at_end=True`) and model-selection metric (accuracy for the tuned no-mitigation reference) . These training settings significantly affect the final model performance.

4.2 Experiments

We executed three experimental checkpoints in sequence: (i) baseline reproduction using MMS-300m model, (ii) untuned mHuBERT run, and (iii) tuned mHuBERT run. This sequence helps us evaluate the impact of model selection and tuning before implementing bias-mitigation techniques.

Run	Acc	Macro-F1	Eval Loss
Assignment baseline (MMS-300m)	0.3224	–	3.0566
mHuBERT untuned	0.4676	0.4142	2.2854
mHuBERT tuned	**0.5270**	**0.4748**	**2.1629**

Table 1: Baseline and improvement experiment summary.

4.3 Results

In baseline reproduction, the MMS-300m model achieved 0.3224 accuracy. The untuned mHuBERT run already exceeds this level with 0.4676, and tuned mHuBERT further improves to 0.5270 accuracy and 0.4748 macro-F1. Compared to untuned mHuBERT, tuning improves the accuracy by +0.0594 and the macro-F1 by +0.0607 and this tuned model is used as the reference for mitigation experiments.

5 Speaker-Bias Mitigation

5.1 Methods

We used both data-centric and model-based mitigation. We also test several intermediate variants (Mitigation 2, 3A, 3B, and 4) to understand the trade-offs.

Data-centric Augmentation Mitigation:

- Mitigation 1: speed/noise ($p = 0.30$, speed 0.97–1.03, noise std 0.0005–0.002).
- Mitigation 2: gentler speed/noise ($p = 0.25$, speed 0.98–1.02, noise std 0.0005–0.0015) with macro-F1 checkpointing.
- Mitigation 3A: revert to M1 augmentation strength with macro-F1 checkpointing and more frequent eval/save steps (100/100).
- Mitigation 3B: milder fallback from M3A ($p = 0.28$, speed 0.975–1.025, noise std max 0.0018).
- Mitigation 4: all data-centric augmentation (speed/noise + pitch shift + spectral masking) with global augmentation gate and t-SNE export.

Model-based Mitigation (DANN):

- Architecture: shared encoder with language and adversarial speaker heads.
- Training logic: a gradient reversal layer forces the encoder to reduce speaker-identifiable information while preserving language-discriminative features.
- Final settings: `speaker_loss_weight=0.1`, `grl_lambda=1.0`, lambda schedule enabled, speaker-head dropout 0.1.
- Run design: augmentation was disabled in the final DANN run to isolate the effect of adversarial mitigation from augmentation effects.

Encoder objective:

$$\mathcal{L}_{enc} = \mathcal{L}_{lang} - \lambda\mathcal{L}_{speaker}.$$

5.2 Experiments

We evaluated mitigation variants against the tuned tuned mHuBERT model on the same dataset split and metrics. Table 2 reports overall performance of the model across all languages, and Table 3 shows the most frequent language confusions pairs across untuned, tuned, mitigation-1, and DANN runs.

5.3 Results

Overall, DANN is the strongest mitigation result: +0.0306 accuracy and +0.0678 macro-F1

Run	Acc	Macro-F1	Eval Loss
Tuned ref (no mitigation)	0.5270	0.4748	2.1629
Mitigation 1	0.5388	0.4868	2.1504
Mitigation 2	0.5279	0.4828	2.1385
Mitigation 3A	0.4979	0.4561	2.1884
Mitigation 3B	0.5276	0.4817	2.1456
Mitigation 4	0.5115	0.4664	2.2127
DANN (final improved)	**0.5576**	**0.5426**	**1.8815**

Table 2: Mitigation experiments (validation split).

Pair	Untuned	Tuned	Mitigation 1	DANN
konkani→marathi	106	97	106	74
sindhi→punjabi	79	103	102	14
hindi→urdu	60	77	68	39
dogri→punjabi	53	76	71	44
bodo→manipuri	39	29	39	45
odia→bengali	26	23	45	10
santali→bengali	54	34	40	10

Table 3: Confusion-pair counts across key runs.

over tuned mHuBERT model (no-mitigation), and +0.0188 accuracy and +0.0558 macro-F1 over Mitigation 1. Data-centric methods were useful but sensitive to augmentation strength. Mitigation 1 was the best data-centric variant, while mixed augmentation (Mitigation 4) underperformed.

Confusion behavior also improved for several commonly confused language pairs. (Table 3), especially `sindhi→punjabi` and `hindi→urdu`. Some pairs such as `konkani→marathi` are still difficult, so the bias is reduced but not completely removed.

Speaker-bias diagnostics show that these improvements are not caused by speaker leakage. The training and validation sets have no overlapping speakers `overlap_speaker_count=0`. Likewise, unseen-speaker accuracy increases from 0.4676 (untuned) to 0.5576 (DANN). The weak-class mean accuracy increased from 0.0227 to 0.3547, indicating an improvement in performance on previously weak classes.

6 Analysis

6.1 Bias Source and Learning Challenges

The bias source in this dataset is the limited speaker diversity in training data. The training set contains (110 speakers over 22 languages) while the validation set contains (681 speakers with 3300 samples), and all validation speakers are unseen during training. This creates a speaker bias problem because the model may learn speaker-specific characteristics such as (voice quality or speaking style) instead of learning the language itself.

Learning from this biased data is difficult for two reasons. **First**, with few speakers per language, the encoder can partially fit speaker characteristics during training even though the validation speakers are different. **Second**, it is harder for the model to accurately distinguish between related languages because many of them have similar prosodic and acoustic patterns. As a result, some languages remain weakly separated in the learned representation.

Evidence from our experiments:

- Untuned reference has near-zero classes (for example maithili 0.0000, konkani 0.0000, sindhi 0.0333).
- Confusion matrix shows strong confusion between certain language pairs (for example konkani→marathi and sindhi→punjabi).
- Validation is fully unseen-speaker (seen-speaker samples = 0, unseen-speaker samples = 3300), so weak performance cannot be explained by train/validation speaker overlap.

6.2 Mitigation Approach and Results

Our approach combines data-centric mitigation and model-based adversarial de-biasing to reduce speaker bias.

- Data-centric mitigation (especially Mitigation 1) improves robustness by breaking fixed speaker-style dependence.
- DANN directly penalizes speaker-identifiable features in the shared encoder.

The results indicate strong but partial improvement. The overall results are shown in Table 4.

Comparison	ΔAcc	ΔMacro-F1
Untuned → DANN	+0.0900	+0.1284
Tuned no-mitigation → DANN	+0.0306	+0.0678
Mitigation 1 → DANN	+0.0188	+0.0558

Table 4: Global impact of DANN versus reference runs.

Class-level behavior shows both improvements and tradeoffs: unseen-speaker accuracy improves from 0.4676 to 0.5576, weak-class mean accuracy increases from 0.0227 to 0.3547. However, some previously strong classes still regress (for example nepali and punjabi vs Mitigation 1). Overall, the mitigation methods reduce speaker bias and improve performance on weak languages, but they do not completely remove speaker-related information.

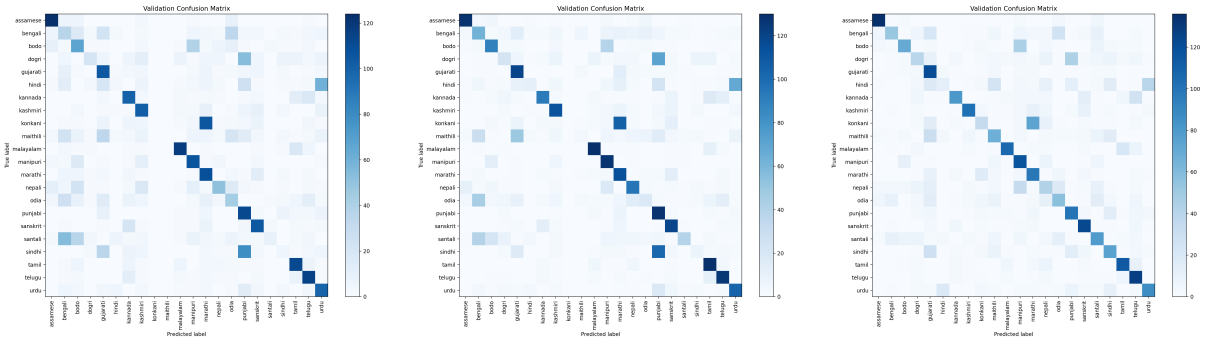

Figure 1: Validation confusion matrices: untuned reference (left), Mitigation 1 (middle), DANN improved (right).

6.3 Confusion Patterns

Most frequent confused langugaes are:

- `konkani→marathi`
- `sindhi→punjabi`
- `hindi→urdu`

These confusions occur mainly because the languages share similar linguistic and acoustic characteristics. As shown in Table 3, DANN reduces dominant confusions, but not all.

Pattern-specific interpretation:

- `hindi→urdu`: similar words and pronunciation patterns in conversational speech.
- `konkani→marathi`: regional similarity and overlapping speaking styles.
- `sindhi→punjabi`: difficult boundary under uneven class difficulty
- `odia→bengali` and `santali→bengali`: neighboring-language proximity

The results also show that DANN reduces dominant confusions compared to the untuned mHuBERT model including (`sindhi→punjabi`, `santali→bengali`, `odia→bengali`, `hindi→urdu`, `konkani→marathi`). Consequently, some alternative confusion channels increase (for example `bodo→manipuri` and `nepali→manipuri` when compared with Mitigation 1), indicating that the model still struggles with closely related languages.

6.4 Representation Analysis (t-SNE)

t-SNE is applied to last-layer validation embeddings.

To ensure consistent comparison, the analysis target setup is fixed (`max_samples`=2200, `perplexity`=30, `seed`=42, `speaker_top_k`=12).

For DANN:

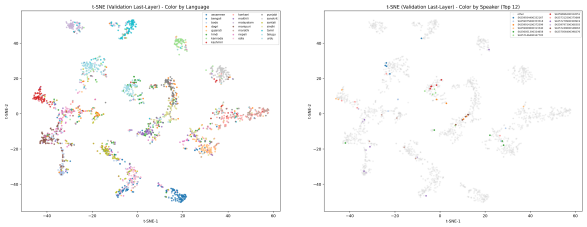

Figure 2: DANN t-SNE on validation embeddings: by language (left), by speaker (right).

- Silhouette (language): 0.0047
- Silhouette (speaker top-k): -0.0992
- kNN speaker purity@5: 0.3433

Interpretation: speaker structure is weakened but not completely removed. The negative speaker silhouette that the model no longer groups samples strongly by speaker, while non-zero kNN speaker purity shows that some speaker identity still remains in embedding neighborhoods.

This result is also supported by other diagnostics:

- `overlap_speaker_count=0`, `warnings=[]` (train and validation speakers are unique)
- unseen-speaker accuracy improves across runs (`0.4676` → `0.5270` → `0.5388` → `0.5576`, with DANN achieving the best performance.)

So, the improved model is more speaker-robust and language-discriminative than earlier mitigations, but it still encodes some speaker information.

7 Conclusion

Tuned mHuBERT provides a strong reference model without bias mitigation. The results show that mitigation performance depends on the training setup. Mild data augmentation improves the model, while very strong augmentation can reduce performance. The best final model is DANN, which

achieves the highest accuracy and improves performance on weaker languages under unseen-speaker validation. The analysis shows that speaker bias is reduced, but some difficult language confusions and speaker-related signals still remain.

References

Badr et al. 2026. Nnti dataset full. https://huggingface.co/datasets/badrex/nnti-dataset-full. Hugging Face dataset card, accessed 2026-03-05.

Alexei Baevski, Henry Zhou, Abdelrahman Mohamed, and Michael Auli. 2020. wav2vec 2.0: A framework for self-supervised learning of speech representations. In *Advances in Neural Information Processing Systems*.

Yaroslav Ganin and Victor Lempitsky. 2014. Unsupervised domain adaptation by backpropagation. https://arxiv.org/abs/1409.7495. ArXiv:1409.7495.

Wei-Ning Hsu, Benjamin Bolte, Yao-Hung Hubert Tsai, Kushal Lakhotia, Ruslan Salakhutdinov, and Abdelrahman Mohamed. 2021. Hubert: Self-supervised speech representation learning by masked prediction of hidden units. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 29:3451–3460.

John D. Hunter. 2007. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95.

Meta AI (facebook). 2026. facebook/mms-300m model card. https://huggingface.co/facebook/mms-300m. Hugging Face model card, accessed 2026-03-07.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*.

Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830.

Vineel Pratap, Andros Tjandra, Bowen Shi, Paden Tomasello, Arun Babu, Souvik Kundu, Ali Elkahky, Zhaoheng Ni, Apoorv Vyas, Ankur Bapna, et al. 2023. Scaling speech technology to 1,000+ languages. *arXiv preprint arXiv:2305.13516*.

PyTorch Audio Team. 2026. Audio data augmentation tutorial. https://docs.pytorch.org/audio/stable/tutorials/audio_data_augmentation_tutorial.html. PyTorch Audio documentation, accessed 2026-03-06.

Laurens van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-sne. *Journal of Machine Learning Research*, 9(86):2579–2605.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, and Jamie Brew. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45.